

```
!pip install pyspark
```

Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/pythc

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("RDD_Practice") \
    .master("local[*]") \
    .getOrCreate()
```

```
sc = spark.sparkContext
```

```
from google.colab import files
```

```
uploaded = files.upload()
```

archive.zip

archive.zip(application/x-zip-compressed) - 45212625 bytes, last modified: 7/23/2026 - 100% done

Saving archive.zip to archive.zip

```
import zipfile
import os

# Assuming the uploaded file is named 'archive.zip'
zip_file_path = 'archive.zip'

# Check if the zip file exists
if os.path.exists(zip_file_path):
    with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
        zip_ref.extractall('.') # Extract all contents to the current directory
        print(f"Successfully unzipped '{zip_file_path}'")
        print("Extracted files:")
        for filename in zip_ref.namelist():
            print(f"- {filename}")
else:
    print(f"Error: Zip file '{zip_file_path}' not found.")
```

Successfully unzipped 'archive.zip'
Extracted files:
- faker_employee.csv

```
rdd = sc.textFile("/content/faker_employee.csv")
```

```
rdd.take(5)
```

```
['Employee ID,First Name,Last Name,Department,Salary,Joining Date,Email  
ID,Address',  
 '1,Nicholas,Martin,Sales,50819,2022-11-16,dannymays@example.com,"14632 Ashley  
Trafficway Suite 345',  
 'Lake Katrinaside, MH 96450"',  
 '2,Joy,Miller,Legal,58024,2023-02-03,heather59@example.net,"00670 Oliver  
Harbors',  
 'Port Andrewchester, TN 14223"']
```

```
header = rdd.first()
```

```
data = rdd.filter(lambda row: row != header)
```

```
employee_rdd = data.map(lambda row: row.split(","))
```

```
employee_rdd.take(3)
```

```
[[ '1',  
  'Nicholas',  
  'Martin',  
  'Sales',  
  '50819',  
  '2022-11-16',  
  'dannymays@example.com',  
  '"14632 Ashley Trafficway Suite 345',  
  'Lake Katrinaside', ' MH 96450"'],  
 [ '2',  
  'Joy',  
  'Miller',  
  'Legal',  
  '58024',  
  '2023-02-03',  
  'heather59@example.net',  
  '"00670 Oliver Harbors"']]
```

```
employee_rdd.count()
```

```
2000000
```

```
employee_rdd.first()
```

```
[ '1',  
  'Nicholas',  
  'Martin',  
  'Sales',  
  '50819',
```

```
'2022-11-16',  
'dannymays@example.com',  
'"14632 Ashley Trafficway Suite 345"]
```

```
header
```

```
'Employee ID,First Name,Last Name,Department,Salary,Joining Date,Email ID,Address'
```

```
bangalore = employee_rdd.filter(lambda x: x[5] == "Bangalore")  
  
#bangalore.collect()
```

```
# Remove the header  
header = rdd.first()  
  
# Keep only data rows  
data_rdd = rdd.filter(lambda row: row != header)  
  
# Split each row into columns  
split_rdd = data_rdd.map(lambda row: row.split(","))  
  
# Filter Salary > 50000  
filtered_rdd = split_rdd.filter(lambda row: int(row[4]) > 50000)  
  
# Print  
for row in filtered_rdd.collect():  
    print(row)
```



```
-----
Py4JJavaError                                Traceback (most recent call last)
/tmp/ipykernel_1224/3302735668.py in <cell line: 0>()
    12
    13 # Print
--> 14 for row in filtered_rdd.collect():
    15     print(row)
```

3 frames

```
/usr/local/lib/python3.12/dist-packages/py4j/protocol.py in
get_return_value(answer, gateway_client, target_id, name)
    325         value = OUTPUT_CONVERTER[type](answer[2:], gateway_client)
    326         if answer[1] == REFERENCE_TYPE:
--> 327             raise Py4JJavaError(
    328                 "An error occurred while calling {0}{1}{2}.\n".
    329                 format(target_id, ".", name), value)
```

```
Py4JJavaError: An error occurred while calling
z:org.apache.spark.api.python.PythonRDD.collectAndServe.
: org.apache.spark.SparkException: Job aborted due to stage failure: Task 0 in
stage 18.0 failed 1 times, most recent failure: Lost task 0.0 in stage 18.0
(TID 36) (2a7142bddd77 executor driver):
org.apache.spark.api.python.PythonException: Traceback (most recent call
last):
  File "/usr/local/lib/python3.12/dist-
packages/pyspark/python/lib/pyspark.zip/pyspark/worker.py", line 2044, in main
process()
  File "/usr/local/lib/python3.12/dist-
packages/pyspark/python/lib/pyspark.zip/pyspark/worker.py", line 2036, in
process
    serializer.dump_stream(out_iter, outfile)
  File "/usr/local/lib/python3.12/dist-
packages/pyspark/python/lib/pyspark.zip/pyspark/serializers.py", line 273, in
dump_stream
    vs = list(itertools.islice(iterator, batch))
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-
packages/pyspark/python/lib/pyspark.zip/pyspark/util.py", line 131, in wrapper
    return f(*args, **kwargs)
    ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/tmp/ipykernel_1224/3302735668.py", line 11, in <lambda>
IndexError: list index out of range
```

```
at
org.apache.spark.api.python.BasePythonRunner$ReaderIterator.handlePythonExcept
at
org.apache.spark.api.python.PythonRunner$$anon$3.read(PythonRunner.scala:940)
at
org.apache.spark.api.python.PythonRunner$$anon$3.read(PythonRunner.scala:925)
at
org.apache.spark.api.python.BasePythonRunner$ReaderIterator.hasNext(PythonRunn
at
org.apache.spark.InterruptibleIterator.hasNext(InterruptibleIterator.scala:37)
at scala.collection.mutable.Growable.addAll(Growable.scala:61)
at scala.collection.mutable.Growable.addAll$(Growable.scala:57)
at scala.collection.mutable.ArrayBuilder.addAll(ArrayBuilder.scala:75)
```

```

    at scala.collection.IterableOnceOps.toArray(IterableOnce.scala:1505)
    at scala.collection.IterableOnceOps.toArray$(IterableOnce.scala:1498)
    at
org.apache.spark.InterruptibleIterator.toArray(InterruptibleIterator.scala:28)
    at org.apache.spark.rdd.RDD.$anonfun$collect$2(RDD.scala:1057)
    at
org.apache.spark.SparkContext.$anonfun$runJob$5(SparkContext.scala:2524)
    at org.apache.spark.scheduler.ResultTask.runTask(ResultTask.scala:93)
    at
org.apache.spark.TaskContext.runTaskWithListeners(TaskContext.scala:171)
    at org.apache.spark.scheduler.Task.run(Task.scala:147)
    at
org.apache.spark.executor.Executor$TaskRunner.$anonfun$run$5(Executor.scala:64)
    at
org.apache.spark.util.SparkErrorUtils.tryWithSafeFinally(SparkErrorUtils.scala
    at
org.apache.spark.util.SparkErrorUtils.tryWithSafeFinally$(SparkErrorUtils.scal
    at org.apache.spark.util.Utils$.tryWithSafeFinally(Utils.scala:99)
    at org.apache.spark.executor.Executor$TaskRunner.run(Executor.scala:650)
    at
java.base/java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor
    at
java.base/java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecuto
    at java.base/java.lang.Thread.run(Thread.java:840)

```

Driver stacktrace:

```

    at
org.apache.spark.scheduler.DAGScheduler.$anonfun$abortStage$3(DAGScheduler.sca
    at scala.Option.getOrElse(Option.scala:201)
    at
org.apache.spark.scheduler.DAGScheduler.$anonfun$abortStage$2(DAGScheduler.sca
    at
org.apache.spark.scheduler.DAGScheduler.$anonfun$abortStage$2$adapted(DAGSched
    at scala.collection.immutable.List.foreach(List.scala:334)
    at
org.apache.spark.scheduler.DAGScheduler.abortStage(DAGScheduler.scala:2927)
    at
org.apache.spark.scheduler.DAGScheduler.$anonfun$handleTaskSetFailed$1(DAGSche
    at
org.apache.spark.scheduler.DAGScheduler.$anonfun$handleTaskSetFailed$1$adapted
    at scala.Option.foreach(Option.scala:437)
    at
org.apache.spark.scheduler.DAGScheduler.handleTaskSetFailed(DAGScheduler.scala
    at
org.apache.spark.scheduler.DAGSchedulerEventProcessLoop.doOnReceive(DAGSchedul
    at
org.apache.spark.scheduler.DAGSchedulerEventProcessLoop.onReceive(DAGScheduler
    at
org.apache.spark.scheduler.DAGSchedulerEventProcessLoop.onReceive(DAGScheduler
    at org.apache.spark.util.EventLoop$$anon$1.run(EventLoop.scala:50)
    at org.apache.spark.scheduler.DAGScheduler.runJob(DAGScheduler.scala:1009)
    at org.apache.spark.SparkContext.runJob(SparkContext.scala:2484)
    at org.apache.spark.SparkContext.runJob(SparkContext.scala:2505)
    at org.apache.spark.SparkContext.runJob(SparkContext.scala:2524)
    at org.apache.spark.SparkContext.runJob(SparkContext.scala:2549)
    at org.apache.spark.rdd.RDD.$anonfun$collect$1(RDD.scala:1057)

```